Book-Manager

Test-Driven Development Project Report
Advanced Programming Technique
6-Credit Exam

Prof. Lorenzo Bettini

Muhammed Irfan Cheriya Puthan Veettil
Matriculation No: 7127908
muhammed.cheriya@edu.unifi.it
Project Repository: https://github.com/Irfancpv99/BookManager
June 8, 2026

Contents

1 Project Overview and Motivation

This report documents the development of Book Manager, a desktop book management application built using Test-Driven Development (TDD) principles. The application provides a complete book management system where users can create, update, and delete books, each identified by a title, author, and category.

1.1 What This Application Does

The application manages a personal book library organized by categories. Users can add books, edit their details, delete them, and associate each book with a category (Fiction, Non-Fiction, Science, History, Technology). The entire interface runs as a desktop application built with Java Swing. The application uses MongoDB as its persistence layer, providing document storage for books and categories. On startup, the application automatically initializes five default categories if the database is empty, handled by the DatabaseInitializer class.

1.2 TDD Methodology

The project follows the Red-Green-Refactor cycle at every level of development. Tests were written before implementation code, ensuring that each feature was validated against explicit behavioral specifications. This approach resulted in a multi-level test suite with three distinct layers

Unit tests verify individual components in isolation using mock dependencies. These tests focus on business logic within the service layer, controller behavior, and model validation. Mockito provides the mocking framework to ensure tests remain fast and independent of external systems.

Integration tests validate interactions between application layers using real MongoDB instances provided by Docker. These tests ensure that MongoDB document mappings, repository queries, and service layer interactions function correctly.

End-to-end tests exercise the complete application stack, including database layer, service layer, and the Java Swing desktop interface. AssertJ Swing tests verify user workflows through simulated GUI interactions, validating complete scenarios such as creating books, editing them, deleting and handling error cases.

2 Technical Architecture

2.1 System Structure

The application is organised into four layers. Each layer communicates only with its next layers.

Presentation Layer - The GUI built with Java Swing. BookSwingView contains the book list, text fields for title and author, a category combo box, and three action buttons (Add Book, Save Edit, Delete Selected). No business logic lives here; it only handles user interaction and delegates all actions to the controller via named component events. Components are named explicitly (e.g., "titleField", "bookList") to support reliable GUI testing.

Controller Layer -BookController sits between the GUI and business logic. When users click "Add Book" or "Save Edit", the controller delegates directly to the service, passing the current view as the callback target. It never touches repositories directly.

Service Layer - BookService handles all business logic. It validates inputs (non-empty title, non-empty author, non-null category), enforces uniqueness on add, checks existence on update and delete, and dispatches results back to the view. The DatabaseInitializer service handles automatic category seeding on first startup.

Repository Layer - BookRepository and CategoryRepository are interfaces defining CRUD operations. MongoDB implementations (MongoBookRepository, MongoCategoryRepository) handle actual database interactions using the MongoDB Java Driver.

2.2 Domain Model

Book has four fields: id, title, author, and categoryId. Title and author cannot be null or empty — the service layer enforces these rules and reports validation errors via the view. The id is assigned at the time of creation in the view layer using UUID.randomUUID(). Equality and hash code are based solely on id.

Category has two fields: id and name. The five default categories (cat-1 through cat-5) are seeded by DatabaseInitializer if the categories collection is empty. Equality and hash code are based solely on id. Both domain classes implement proper equals() and hashCode() methods. toString() is overridden: Book returns "title - author" and Category returns its name, making them suitable for direct use in Swing list and combo box renderers.

2.2.1 Entity Relationship

The domain model consists of two entities: `Book` and `Category`, with a many-to-one relationship in which each `Book` belongs to exactly one `Category`.

In the MongoDB implementation, each book document stores a `categoryId` field containing the string ID of the corresponding category. This follows MongoDB's document-oriented model, where references between collections are stored as field values rather than enforced through foreign key constraints. The `BookSwingView` resolves the display by populating the `JComboBox<Category>` with live category objects and matching `categoryId` on book selection.

```
Auther Name - Book Name | Category
```

2.3 Repository Pattern

Data access is abstracted behind clean interfaces. `BookRepository` defines `save`, `update`, and `delete`. CategoryRepository defines `save`, `update`, and `delete`. MongoDB implementations use the MongoDB Java Driver directly, with private helper methods `(toDocument, fromDocument)` mapping between domain objects and BSON documents.

2.4 Technology Stack

Table 1 lists the complete technology stack.

Purpose	Technology
Language / Build	Java 17 + Maven
Database	MongoDB 6.0
MongoDB Driver	MongoDB Java Driver 4.11.1
GUI Toolkit	Java Swing
Unit Testing	JUnit 5.10.1 + Mockito 5.8.0
GUI Testing	AssertJ Swing 3.17.1
Code Coverage	JaCoCo 0.8.11
Mutation Testing	PITest 1.14.1
Code Quality	SonarCloud
Coverage Tracking	Codecov
CI/CD	GitHub Actions

Table 1: Technology stack overview

3 Design and Implementation Choices

3.1 Architecture Overview

The application follows the Model–View–Controller (MVC) pattern with an additional service layer between the controller and repositories. This structure matches the approach used in the course book, where the controller handles user input, the service contains business logic, and repositories manage data access.

- **View abstraction:** The view is defined through the `BookView` interface and implemented by `BookSwingView`. This allows the controller and service to be tested with a mocked view, without creating Swing components. If the UI changes, only a new `BookView` implementation is required.
- **Service layer:** The controller does not access repositories directly. Business operations such as creating or deleting books are handled by `BookService`, keeping responsibilities clear and enabling independent testing with mocks.
- **Database initialization:** Default categories (PERSONAL, WORK, STUDY) are initialized in a separate `DatabaseInitializer` class. This improves readability and allows the initialization logic to be tested in isolation.
- **Repository design:** Repositories are defined as interfaces, with MongoDB-specific implementations using the MongoDB Java Driver. Unit tests use mocked repositories, while integration tests run against a real MongoDB started by Docker.
- **Database initialization:** The five default categories are seeded by a dedicated DatabaseInitializer class, which checks whether the categories collection is empty before inserting. This logic is separately unit-tested in `DatabaseInitializerTest`.
- **Docker for testing:** MongoDB is started for integration tests using the `fabric8 docker-maven-plugin`. The container is started before integration tests and stopped afterward as part of the Maven lifecycle.

4 Test-Driven Development in Practice

4.1 The Red-Green-Refactor Workflow

Every feature started with a test. Write the test first, watch it fail (red), write code to make it pass (green), then clean up the code (refactor). This cycle repeated for every method, every class.

4.2 Testing Strategy: The Testing Pyramid

The project has three test levels - lots of fast unit tests at the bottom, some integration tests in the middle, and a few slow end-to-end tests at the top.

Unit Tests Unit tests test one class at a time using mock objects. `BookServiceTest` tests all validation and delegation logic using a mocked `BookRepository`, `CategoryRepository`, and `BookView`. `BookControllerTest` verifies that each controller method delegates correctly to the service. `BookSwingViewTest` tests the full button state machine, form population, list updates, and edit/delete workflows—all without Swing threading—by driving the component directly. `DatabaseInitializerTest` tests the seeding logic in isolation using a mocked `CategoryRepository`.

Integration Tests Integration tests verify that components work together correctly using a real MongoDB instance started by Docker. `MongoBookRepositoryIT` and `MongoCategoryReposi` test all CRUD operations against a live database, with each test class dropping its collection in `@BeforeEach` for isolation. `BookServiceIT` and `BookControllerIT` wire together the full stack below the view, using a `CapturingView` helper to assert on the results delivered to the view.

End-to-End Tests End-to-end tests test the whole application as a user would. `BookManagerE2E` uses AssertJ Swing to simulate clicks, text entry, and list interactions against a fully assembled application with a real MongoDB database (`bookmanager_e2e`). Scenarios cover startup state, adding books, editing books, deleting books, and verifying persistence directly in the MongoDB collection.

4.3 GUI Testing

Desktop GUIs are difficult to test because Swing runs on a dedicated Event Dispatch Thread (EDT). AssertJ Swing handles this complexity by synchronising test actions with the EDT automatically. `GuiActionRunner.execute()` is used to construct the view and wire dependencies on the correct thread. Component names set in code (e.g., `"titleField"`, `"addButton"`, `"bookList"`) allow tests to locate elements reliably regardless of layout changes.

Unit-level view tests (`BookSwingViewTest`) drive `BookSwingView` directly without AssertJ Swing, using `MockitoAnnotations.openMocks` and calling view methods and button actions directly. End-to-end tests use `FrameFixture` from AssertJ Swing for full interaction simulation.

Docker Configuration for Testing

Docker provides a real MongoDB 6.0 instance for integration and end-to-end tests. The `fabric8 docker-maven-plugin` is configured in `pom.xml` with two lifecycle executions:

- `docker-start` bound to `pre-integration-test`: pulls and starts a `mongo:6.0` container on port 27017, waiting up to 20 seconds for the log message `"Waiting for connections"` before proceeding.
- `docker-stop` bound to `post-integration-test`: stops and removes the container after tests complete.

When running `mvn clean verify`, Maven automatically starts MongoDB before integration tests (executed by the Failsafe plugin, which picks up both `*IT.java` and `*E2E.java`) and stops it afterward. Unit tests (executed by the Surefire plugin) do not require Docker since they rely on mocked dependencies.

When running integration or end-to-end tests from Eclipse using *Run As → JUnit Test*, MongoDB must be started manually beforehand:

```
docker start mongodb
```

In GitHub Actions, the Docker Maven Plugin manages the container lifecycle automatically during `mvn clean verify`. GUI tests are executed using `xvfb-run` to provide a virtual display for Swing-based components.

Mutation Testing

PITest verifies test quality by intentionally introducing bugs (mutations) into the production code and checking whether the test suite detects them. It modifies conditions, return values, and method calls, then reruns the tests. If tests still pass after a mutation, they are considered weak. If tests fail, the mutation is killed—confirming that the test suite exercises that behaviour.

The PIT configuration targets `com.bookmanager.service.*` and `com.bookmanager.controller.*` as the primary classes under mutation, with the `DEFAULTS` mutator set. A `mutationThreshold` of `100` is enforced, meaning the build fails if any mutation survives. Model classes are also targeted to verify that `equals`, `hashCode`, `toString`, getters, and setters are fully exercised.

The following example from `BookServiceTest` illustrates a mutation-killing test:

```
@Test
void addBook_duplicateId_showsError() {
    when(bookRepository.findById("book-1")).thenReturn(book1);
    bookService.addBook(new Book("book-1", "1984", "George Orwell",
        "cat-1"), bookView);
    verify(bookView).showError("Book with id book-1 already exists"
       );
    verify(bookRepository, never()).save(any());
}
```

This single test kills multiple mutations: removing the duplicate check, inverting the null condition, and removing the early return all cause it to fail.

5 Exclusions and Justifications

5.1 Code Coverage Exclusions

App.java - Main class containing only the main() method and Swing initialization. As explained in the book, main() methods are bootstrap code that cannot be meaningfully unit tested. The class only creates MongoDB client, repositories, services, controller, view, and initializer - all tested independently. E2E tests cover the complete application startup.

5.2 Mutation Testing Exclusions

- **View Layer** – Mutation testing focuses on business logic. The View contains UI setup code (Swing components, layouts, event listeners) where mutations produce equivalent or non-meaningful changes. View correctness is verified via unit tests with mocks and E2E tests with AssertJSwing.
- **Repository Layer** – Repository classes are infrastructure code delegating to the MongoDB driver. As recommended in the book, repositories should be tested with real databases via integration tests. Mutating repositories would target MongoDB API calls, not business logic. Correctness is verified through integration tests with Docker.
- **App Class** – Bootstrap code containing only the `main()` method with no business logic to mutate.

6 Problem Encountered

6.1 Known Limitation: Java 17 JPMS Compatibility Warning

During GUI testing with AssertJ Swing 3.17.1, the Java 17 module system generates `InaccessibleObjectException` warnings. These warnings occur because AssertJ Swing performs reflective access to `java.util.TimerTask.state` for test synchronization, which is blocked by the Java Platform Module System (JPMS).

Solution were implemented to solve the warning:

- **JVM Module Access Configuration**
  Added `-add-opens java.base/java.util=ALL-UNNAMED` to Maven Failsafe plugins via the `argLine` parameter.

6.2 JUnit 4 and JUnit 5 coexistence

. AssertJ Swing requires JUnit 4 , while the rest of the test suite uses JUnit 5. To run both in a single build, the JUnit Vintage Engine (junit-vintage-engine) was added as a dependency, allowing Maven's Surefire and Failsafe plugins to execute both JUnit 4 and JUnit 5 tests transparently.

6.3 Codecov Integration

JaCoCo was configured to generate a single XML coverage report during the Maven `verify` phase. The report is uploaded to Codecov through the GitHub Actions workflow, allowing coverage metrics to be tracked automatically for every push and pull request. The repository README includes a Codecov badge displaying the current project coverage.

7 Quality Metrics and Results

7.1 Code Coverage with JaCoCo

JaCoCo is used to measure code coverage across all test types, including unit, integration, and end-to-end tests. The `App` class is excluded from coverage requirements, as documented in Section 4.1, since it contains only application bootstrap logic.

Table 2 summarizes the final code coverage results. All included packages achieve 100% coverage across all reported metrics.

Table 2: JaCoCo Code Coverage Summary

Package	**Instructions**	**Branches**	**Lines**	**Methods**
model	100%	100%	100%	100%
service	100%	100%	100%	100%
controller	100%	100%	100%	100%
view.swing	100%	100%	100%	100%
repository.mongo	100%	100%	100%	100%
Total	**100%**	**100%**	**100%**	**100%**

Coverage reports are generated automatically during the Maven `verify` phase and are uploaded to Codecov, which tracks coverage trends over time. The project repository README includes a Codecov badge displaying the current coverage percentage.

7.2 Mutation Testing with PITest

PITest was executed on the `model`, `service`, and `controller` packages, which contain the core business logic of the application. All generated mutations were successfully killed, resulting in a 100% mutation score with zero surviving mutants.

Table 3 presents the mutation testing results aggregated by package.

Table 3: PITest Mutation Coverage by Package

Package	**Line Coverage**	**Mutation Coverage**	**Mutation Strength**
model	100%	100%	100%
service	100%	100%	100%
controller	100%	100%	100%
Total	**100%**	**100%**	**100%**

A mutation score of 100% indicates that every injected fault caused at least one test to fail. This demonstrates that the test suite effectively validates program behavior rather than merely executing code paths.

7.3 Static Analysis with SonarCloud

SonarCloud performs automated static code analysis on every push to the repository. It evaluates code quality by detecting code smells, bugs, security vulnerabilities, and by estimating technical debt. All checks pass the SonarCloud Quality Gate with the highest possible ratings.

.

Table 4 summarizes the SonarCloud analysis results.

Table 4: SonarCloud Quality Gate Summary

Metric	**Value**
Code Smells	0
Bugs	0
Vulnerabilities	0
Security Hotspots	0 (Reviewed)
Technical Debt	0 minutes
Duplications	0%
Maintainability Rating	A
Reliability Rating	A
Security Rating	A

.

The absence of technical debt and quality issues is a direct result of adhering to clean code principles, including meaningful naming, short methods with single responsibilities, consistent formatting, and proper error handling.

7.4 Continuous Integration Pipeline

A continuous integration (CI) pipeline is implemented using GitHub Actions and is triggered on every push to the `main` and `develop` branches, as well as on pull requests targeting the `develop` branch. The pipeline enforces automated quality checks to ensure that only validated code is merged.

The CI pipeline consists of the following stages:

1. **Build and Test with GUI Support**
   The project is built using Maven, and all tests are executed, including unit, integration, and end-to-end tests. Swing-based GUI tests are supported using `xvfb-run` to provide a virtual display environment. A MongoDB instance is started via the Docker Maven Plugin to support repository integration tests.

2. **Mutation Testing**
   PITest is executed against the `model`, `service` and `controller` packages to evaluate the effectiveness of the test suite.

3. **Coverage Reporting**
   Code coverage data generated by JaCoCo is uploaded to Codecov continuous tracking of coverage metrics over time.

4. **Static Analysis**
   SonarCloud performs static code analysis, including detection of code smells, bugs, and security issues, and enforces the project quality gate.

All pipeline stages must complete successfully for the build to pass. Any failure indicates a violation of quality requirements and must be resolved before code can be merged.

8 Branching Strategy and CI Workflow

The project follows a branching workflow based on GitHub Pull Requests to manage feature development and ensure code quality.

The main development branches used were:

- **controller** — Development and implementation of controller classes.
- **category** — Implementation of category related features.
- **service** — Development of service-layer components containing the core application functionality and business rules.
- **view** — Implementation and of the graphical user interface components.
- **ITtest** — Integration tests to verify interactions between multiple components.
- **e2e** — Development of end-to-end tests to validate complete user workflows and system behavior.
- **coverage** — Improvements related to code coverage measurement.

Each feature or task was developed in its own dedicated branch and merged into the **main** branch through a GitHub Pull Request. Once the branch was considered stable and all CI checks had passed, it was merged into the **main** branch. After merging, feature branches were deleted to keep the repository clean and maintain an organized workflow.

The GitHub Actions CI pipeline is configured to run on pushes to the **main** and other branches, as well as on pull requests targeting **main**. This setup ensures that every proposed change is automatically validated before being merged into the main codebase.

9 Running the Project

The project requires Java 17, Maven 3.12.1 , and Docker for running MongoDB during tests and development. First line.

Clone the repository:

```
git clone https://github.com/Irfancpv99/BookManager.git
cd BookManager
```

9.1 Eclipse Setup

Import the project into Eclipse:

1. Open Eclipse IDE
2. Select `File` → `Import` → `Maven` → `Existing Maven Projects`
3. Browse to the cloned `BookManager` directory
4. Select `pom.xml` and click `Finish`

Expected Result: Project imports with no errors or warnings

Verify Tests Run:

- Right-click `src/test/java`
- Right-click `src/it/java`
- Right-click `src/e2e/java`
- Select `Run As` → `JUnit Test`
- All tests should pass

Run Application from Eclipse:

- Right-click `App.java`
- Select `Run As` → `Java Application`
- GUI window should open

Running from Terminal

```
mvn package
docker start mongodb
docker stop mongodb
```

Alternatively, the application can be started using `mvn exec:java`

9.2 Running Tests

Execute only unit tests (faster):

```
mvn clean test
```

Execute all tests (unit, integration, and end-to-end):

```
mvn clean verify
```

Generate the code coverage report:

```
mvn clean verify jacoco:report
View: target/site/jacoco/index.html
```

Run mutation testing:

```
mvn clean test org.pitest:pitest-maven:mutationCoverage
View: target/pit-reports/index.html
```

10 Project Summary

Book-Manager demonstrates how to build software with proper testing practices. The project achieves all quality targets: 100% code coverage (except the main class), 100% mutation coverage, zero technical debt from SonarCloud, and tests at three levels (unit, integration, end-to-end). GitHub Actions runs quality checks automatically on every push.

The MongoDB-based persistence works well for a desktop book management application. The repository pattern provides clean separation between business logic and data access, making the codebase easy to test and maintain.

Test-Driven Development drove every feature. Every piece of code exists because a test required it. The test suite gives confidence to change code without breaking existing functionality. Red-Green-Refactor kept development disciplined throughout the project.

GUI testing with AssertJ Swing proved valuable. These tests caught integration bugs that unit tests missed - like timing issues with the Event Dispatch Thread and state management between view and controller. Using xvfb-run in CI makes GUI tests work in headless environments.

The combination of JaCoCo (100% coverage), PITest (100% mutation coverage), and SonarCloud (zero issues) demonstrates that the tests actually verify behavior. This project shows that good TDD practices create reliable, maintainable software.

Repository: https://github.com/Irfancpv99/BookManager

CI/CD: https://github.com/Irfancpv99/BookManager/actions

Coverage: https://app.codecov.io/gh/Irfancpv99/BookManager

Quality: https://sonarcloud.io/project/overview?id=Irfancpv99_BookManager